

DLLI Linking

בקרית איכות
ובדיקות תוכנה

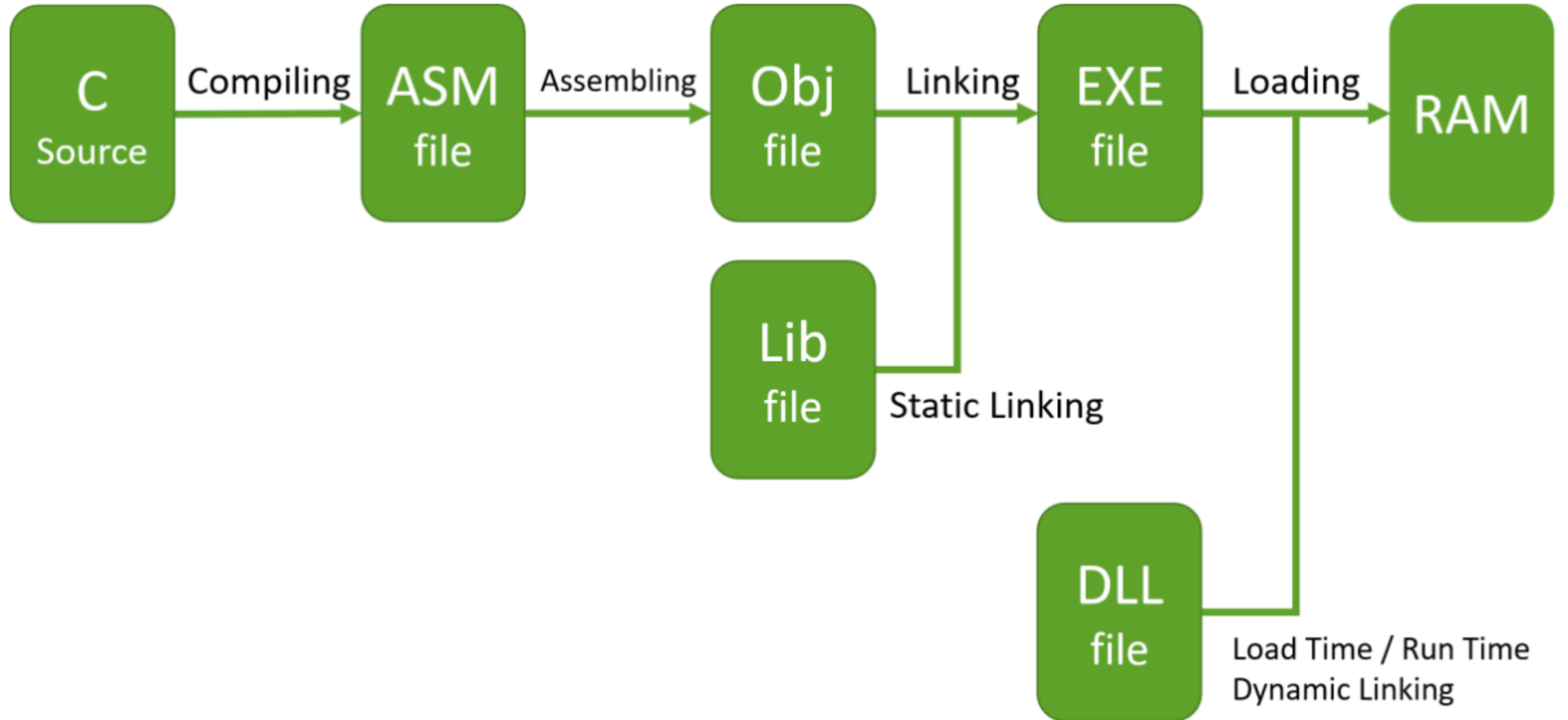
DLL LINKING

בפרקים שדנו בזיכרון התבוננו בזיכרון מנקודת מבטו של המעבד. הבעיה שהצגנו היתה ניתנת לתיאור באופן הבא – "המעבד רוצה לפנות לכתובת מסויימת במרחב הכתובות הוירטואלי שלו, כיצד הכתובת הזו מאותרת ב-RAM?". בשלב זה אנחנו הולכים להחליף את נקודת המבט ולשאול את השאלה "כיצד בכלל יודע המעבד לאיזו כתובת וירטואלית עליו לפנות כדי לגשת לפקודה או למשתנה כלשהם? מה שרשרת התהליכים שגורמת לכך שמשתנה או פקודה כלשהי יופיעו בכתובת מסויימת בזיכרון ב-RAM, ואיך המעבד יודע לאן לגשת?".

עקב מורכבות וריבוי הפרטים, נחלק את התשובה שלנו לשני פרקים. בפרק הנוכחי נפרט אודות תהליך יצירת קבצי הרצה כאשר תוך כדי הלימוד של קבצי הרצה נלמד גם על יצירת קבצי DLL, שכפי שנראה יש להם פורמט זהה. בפרק הבא "פורמט PE ותהליך ה-Loading" נצא מתוך הנחה שיש בידינו קובץ הרצה או קובץ DLL ונבין כיצד הם נטענים לזיכרון.

סיכום התהליך מתואר בדיאגרמה הבאה. נתחיל מצד שמאל של הדיאגרמה ונתקדם שלב שלב עד שנבין את התהליך כולו.

DLL LINKING



DLL LINKING

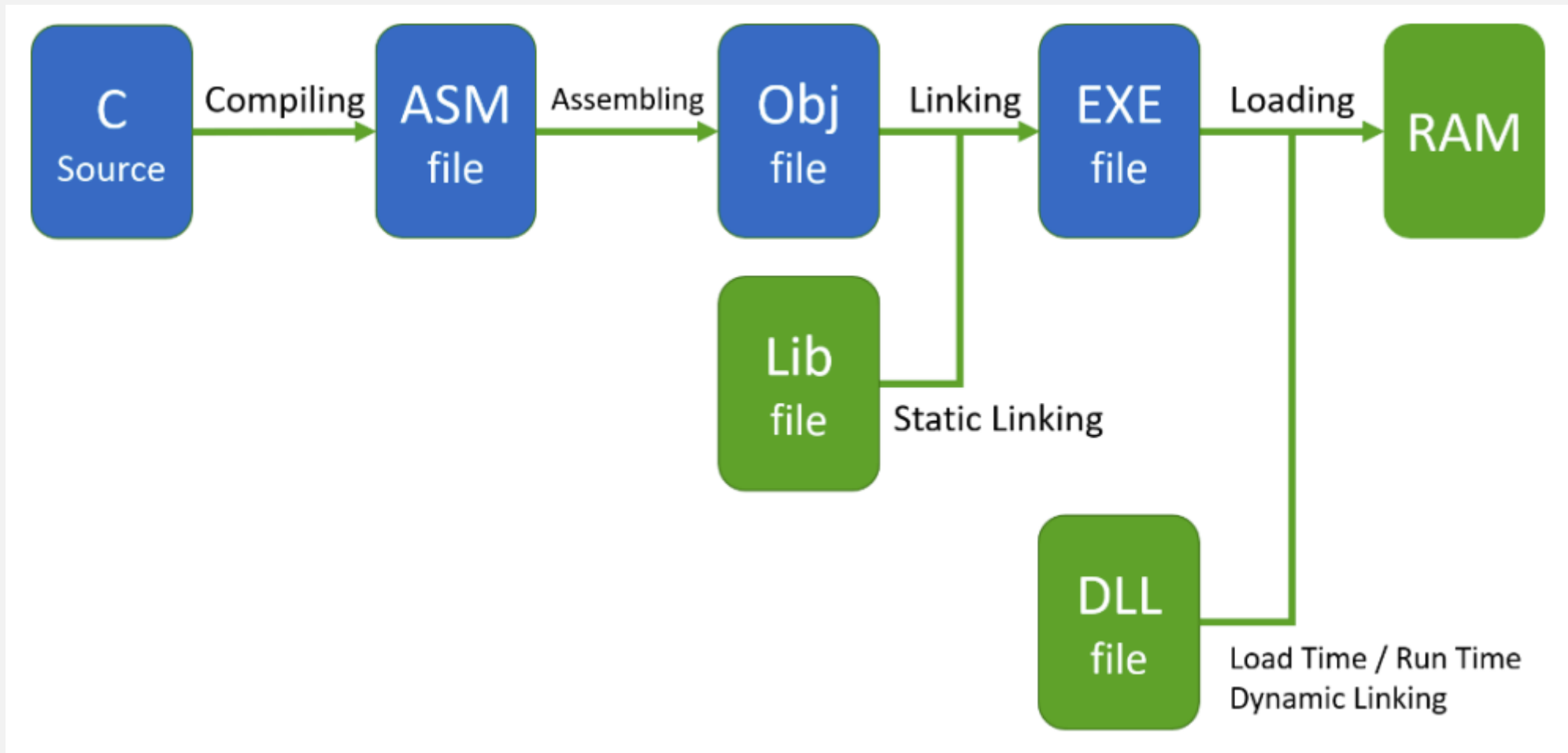
תוך כדי הלימוד נענה על השאלות הבאות:

1. מהם שלבי ההמרה של קבצי מקור בשפת C לקבצי הרצה exe
2. מהי library ומהו Static Linking
3. מהו DLL
4. כיצד לייבא DLL ב-Load Time
5. כיצד לייבא DLL ב-Run Time
6. מהו DLL Injection ואיך אפשר לבצע פעולה זו באמצעות הרגיסטרי
7. לבסוף, נסקור שיטה שמבצעת DLL Injection באמצעות פתיחת thread בתוך process אחר

תהליך המרת קוד משפת C לקובץ הרצה
EXE

תהליך המרת קוד משפת C לקובץ הרצה EXE

המטרה שלנו היא להבין את השינויים שעובר הקוד שלנו בדרכו להפוך לשפת מכונה, נתמקד בכתובות בזיכרון בהן נשמרים המשתנים והקוד שלנו. ההסבר שלנו יכלול את השלבים הצבועים בכחול:



תהליך המרת קוד משפת C לקובץ הרצה EXE

תהליך ה-Compiling

נתחיל את התהליך משני קבצי C.

הקובץ הראשון, main_file.c, כולל את הקוד הבא:

```
#include <stdio.h>
#include "my_funcs.h"

int main(){
    static int a = 2;
    static int b = 3;
    int c;

    c = my_add(a, b);
    printf("%d", c);
    return 0;
}
```

תהליך המרת קוד משפת C לקובץ הרצה EXE

נוח לצפות בתוצרי תהליך ה-Compiling כאשר משתמשים בקומפיילר gcc של MinGW (סביבת קוד פתוח עבור windows). להלן הקוד של פונקציית ה-main, הפעם בשפת אסמבלי:

```
#include <stdio.h>
#include "my_funcs.h"

int main(){
    static int a = 2;
    static int b = 3;
    int c;
    c = my_add(a, b);
    printf("%d", c);
    return 0;
}
```

```
movl    _b.2064, %edx
movl    _a.2063, %eax
movl    %edx, 4(%esp)
movl    %eax, (%esp)
call    _my_add
movl    %eax, 28(%esp)
movl    28(%esp), %eax
movl    %eax, 4(%esp)
movl    $LC0, (%esp)
call    _printf
movl    $0, %eax
leave
```

תהליך המרת קוד משפת C לקובץ הרצה EXE

```
movl    _b.2064, %edx    movl    28(%esp), %eax
movl    _a.2063, %eax    movl    %eax, 4(%esp)
movl    %edx, 4(%esp)    movl    $LC0, (%esp)
movl    %eax, (%esp)    call    _printf
call    _my_add          movl    $0, %eax
movl    %eax, 28(%esp)    leave
```

למי מכם ששולט באסמבלי, שימו לב לכך שסדר האופרנדים הפוך מהרגיל. לדוגמה בשורה הראשונה `_b.2064` מועתק לתוך `edx`, ולא להיפך. זהו תחביר אסמבלי של חברת AT&T. למי מכם שאינו שולט באסמבלי, הסבר קצרצר. הקוד מחולק ל-3 חלקים. החלק הראשון כולל את הקוד עד `call my_add`, החלק השני כולל את הקוד עד `call printf` והחלק השלישי הוא סיום ריצת ה-`main` וביצוע `return 0`.

תהליך המרת קוד משפת C לקובץ הרצה EXE

```
movl    _b.2064, %edx    movl    28(%esp), %eax
movl    _a.2063, %eax    movl    %eax, 4(%esp)
movl    %edx, 4(%esp)    movl    $LC0, (%esp)
movl    %eax, (%esp)    call    _printf
call    _my_add          movl    $0, %eax
movl    %eax, 28(%esp)    leave
```

בכל אחד משני החלקים הראשונים מתבצעת העברה של הפרמטרים לתוך הפונקציה. לדוגמה במקרה של `my_add`, יש להעביר לה את הפרמטרים `a`, `b`. לאחר ריצת כל אחת מהפונקציות, ערך החזרה של הפונקציה נכנס לתוך הרגיסטר `eax`. השימוש ברגיסטר `eax` להחזרת ערך התוצאה של פונקציה הוא מוסכמה מקובלת בקריאה לפונקציות, מה שנקרא Calling Convention (הסבר מפורט בספר האסמבלי של המרכז לחינוך סייבר). כך, לאחר סיום ריצת הפונקציה `my_add`, רואים שמופיעות שורות קוד עם הרגיסטר `eax`, שמעביר את תוצאת החישוב אל המשתנה המקומי שהוגדר.

תהליך המרת קוד משפת C לקובץ הרצה EXE

```
movl    _b.2064, %edx      movl    28(%esp), %eax
movl    _a.2063, %eax      movl    %eax, 4(%esp)
movl    %edx, 4(%esp)      movl    $LC0, (%esp)
movl    %eax, (%esp)       call     _printf
call     _my_add           movl    $0, %eax
movl    %eax, 28(%esp)     leave
```

כפי שבטח שמתם לב, שמות הפונקציות והמשתנים כוללים קו תחתי (Underscore). הסיבה לכך היא שיכול להיות שמתכנת בשפה עילית קרא לפונקציה או למשתנה בשם של פקודת אסמבלי. לדוגמה אפשר לקרוא לפונקציה בשם "call" או "leave". אילו הקומפיילר היה משאיר את השמות כפי שהם, הדבר היה עלול לגרום בלבול בשלב הבא, בו מבוצעת ההמרה משפת אסמבלי לשפת מכונה. לכן הקומפיילר מוסיף _ לפני כל משתנה או פונקציה, כך שיהיה ברור שזו אינה פקודת אסמבלי.

תהליך המרת קוד משפת C לקובץ הרצה EXE

```
movl    _b.2064, %edx    movl    28(%esp), %eax
movl    _a.2063, %eax    movl    %eax, 4(%esp)
movl    %edx, 4(%esp)    movl    $LC0, (%esp)
movl    %eax, (%esp)    call    _printf
call    _my_add          movl    $0, %eax
movl    %eax, 28(%esp)    leave
```

המשתנים a ו-b הוחלפו בסימבולים _a.2063 ו-_b.2064. הסימבולים הללו מציינים שיש להם כתובת בזיכרון, אך כרגע הכתובת אינה ידועה. בהמשך התהליך יהיה צורך לפענח את הסימבולים הללו ולהחליף אותם בכתובות בזיכרון. שימו לב לכך שהמשתנה c, שלא הוגדר כסטטי ולכן נשמר על המחסנית, לא דורש סימבול. המשתנה הזה נשמר בכתובת שהיא מיקום יחסי מתחילת המחסנית ולכן הכתובת שלו מבוטאת יחסית לרגיסטר המחסנית esp.

תהליך המרת קוד משפת C לקובץ הרצה EXE

הקובץ השני, my_funcs.c, כולל את ההגדרה של הפונקציה my_add:

```
int my_add(int a, int b)
{
    static int one = 1;
    return a+b+one;
}
```

כאשר נבצע לו Compiling, נקבל את קוד האסמבלי הבא:

```
movl    8(%ebp), %edx
movl    12(%ebp), %eax
addl    %eax, %edx
movl    _one.1554, %eax
addl    %edx, %eax
```

גם במקרה זה, המשתנה הסטטי one מקבל סימבול ולא כתובת בזיכרון. כדי שהמעבד יוכל לפנות למשתנה הזה, הוא יהיה צריך לקבל כתובת.

תהליך המרת קוד משפת C לקובץ הרצה EXE

תהליך ה-Assembling

נעבור לשלב ה-Assembling. בשלב זה נוצר מקוד האסמבלי קוד בשפת מכונה, ששמור בקובץ עם הסיומת .obj. זהו קובץ בפורמט בינארי, שמתאים בדרך כלל למערכת ההפעלה שיצרה אותו, אך עדיין אינו בשל להרצה, כפי שמיד נראה.

נפתח את הקובץ באמצעות תוכנת IDA

seg000:00000112	8B 15 00 00 00 00	mov	edx, ds:dword_0
seg000:00000118	A1 04 00 00 00	mov	eax, ds:dword_4
seg000:0000011D	89 54 24 04	mov	[esp+4], edx
seg000:00000121	89 04 24	mov	[esp], eax
seg000:00000124	E8 00 00 00 00	call	\$+5
seg000:00000129	89 44 24 1C	mov	[esp+1Ch], eax
seg000:0000012D	8B 44 24 1C	mov	eax, [esp+1Ch]
seg000:00000131	89 44 24 04	mov	[esp+4], eax
seg000:00000135	C7 04 24 00 00 00 00	mov	dword ptr [esp], 0
seg000:0000013C	E8 00 00 00 00	call	\$+5
seg000:00000141	B8 00 00 00 00	mov	eax, 0
seg000:00000146	C9	leave	

```
static int a = 2;
static int b = 3;
int c;
c = my_add(a, b);
printf("%d", c);
return 0;
```

seg000:00000112	8B 15 00 00 00 00	mov	edx, ds:dword_0	static int a = 2;
seg000:00000118	A1 04 00 00 00	mov	eax, ds:dword_4	static int b = 3;
seg000:0000011D	89 54 24 04	mov	[esp+4], edx	int c;
seg000:00000121	89 04 24	mov	[esp], eax	c = my_add(a, b);
seg000:00000124	E8 00 00 00 00	call	\$+5	printf("%d", c);
seg000:00000129	89 44 24 1C	mov	[esp+1Ch], eax	return 0;
seg000:0000012D	8B 44 24 1C	mov	eax, [esp+1Ch]	
seg000:00000131	89 44 24 04	mov	[esp+4], eax	
seg000:00000135	C7 04 24 00 00 00 00	mov	dword ptr [esp], 0	
seg000:0000013C	E8 00 00 00 00	call	\$+5	
seg000:00000141	B8 00 00 00 00	mov	eax, 0	
seg000:00000146	C9	leave		

מהן הכתובות בזיכרון של סגמנט הקוד שלנו? בשלב זה הכתובות נראות מוזרות למדי, בין הכתובות 0x112 ל-0x146 מתחילת סגמנט הקוד. חשוב להבין על מה אנחנו מתבוננים כרגע. פתחנו קובץ שנמצא על הדיסק הקשיח, אנחנו מתבוננים בפקודות בשפת מכונה שהכתובות שלהן בזיכרון לא מייצגות את מה שיקרה כאשר תבוצע טעינה של הקוד לזיכרון. בשפה המקצועית, מה שאנחנו מבצעים הינו Static Analysis, כלומר ניתוח קובץ קוד שנמצא על הדיסק. זאת בניגוד ל-Dynamic Analysis, שהוא ניתוח של קוד שנטען ל-RAM. בהמשך נגיע גם לכך.

seg000:00000112	8B 15 00 00 00 00	mov	edx, ds:dword_0
seg000:00000118	A1 04 00 00 00	mov	eax, ds:dword_4
seg000:0000011D	89 54 24 04	mov	[esp+4], edx
seg000:00000121	89 04 24	mov	[esp], eax
seg000:00000124	E8 00 00 00 00	call	\$+5
seg000:00000129	89 44 24 1C	mov	[esp+1Ch], eax
seg000:0000012D	8B 44 24 1C	mov	eax, [esp+1Ch]
seg000:00000131	89 44 24 04	mov	[esp+4], eax
seg000:00000135	C7 04 24 00 00 00 00	mov	dword ptr [esp], 0
seg000:0000013C	E8 00 00 00 00	call	\$+5
seg000:00000141	B8 00 00 00 00	mov	eax, 0
seg000:00000146	C9	leave	

```
static int a = 2;
static int b = 3;
int c;
c = my_add(a, b);
printf("%d", c);
return 0;
```

האזורים המודגשים באדום הם הכתובות בזיכרון אליהם תורגמו המשתנים הסטטיים והפונקציות. מה מופיע לפנינו? המשתנה a נמצא בכתובת בזיכרון 0. המשתנה b נמצא בכתובת 4. הפונקציות my_add ו-printf נמצאות שתיהן בכתובת 0 בזיכרון. המחרוזת "%d" נמצאת אף היא בכתובת 0 (כפי שרואים בשורת הקוד בכתובת 0x135). כמובן שהדבר אינו אפשרי. אפשר להסיק מכך שאומנם הקוד שלנו עבר לשפת מכונה, אבל אף אחת מהכתובות שרשומות למעבד לגשת אליהן איננה נכונה.

תהליך המרת קוד משפת C לקובץ הרצה EXE

איזה מידע יש בקובץ obj?

1. מידע כללי לגבי הקובץ- שם, גודל, זמן יצירה וכו'
2. קוד בינארי – פקודות בשפת מכונה, שהינן התרגום של פקודות האסמבלי שהתקבלו לאחר הקומפילציה
3. מידע לגבי Relocation- זהו מונח חדש שנשתמש בו גם בהמשך. כפי שראינו, הקוד הבינארי עדיין לא מכיל כתובות הגיוניות בזיכרון. כל מקום שיש בו כתובת בזיכרון צריך להיות מתוקן בשלב ה-Linking, והתהליך של שינוי כתובת בזיכרון נקרא Relocation, קצת בדומה לשינוי כתובת מגורים בעולם האמיתי. פעולת תיקון של כתובת ספציפית נקראת לעיתים גם "fix up". כדי שהלינקר יידע אילו כתובות צריך לעבור Relocation, קובץ ה-obj מכיל טבלה עם המידע.

תהליך המרת קוד משפת C לקובץ הרצה EXE

איזה מידע יש בקובץ obj?

4. טבלת סימבולים Symbols- הטבלה כוללת את שמות הסימבולים הגלובליים שהוגדרו בקובץ המקור

ואת שמות הסימבולים שצריך לייבא מקבצים חיצוניים. במקרה שלנו, המשתנה `a` הוא סימבול שמוגדר בקובץ המקור, ואילו הפונקציה `my_add` היא סימבול שמיובא לקוד שלנו.

5. מידע לטובת דיבוג – לדוגמה, שמות של משתנים ושל פונקציות. כפי שראינו בעבר ומיד נראה שוב,

כאשר אנחנו פותחים באמצעות תוכנת `ida` קובץ `exe` שיצרנו, מוצגים לפנינו השמות של הפונקציות לעומת זאת, ללא מידע לטובת דיבוג, נקבל רק כתובות בזיכרון.

תהליך המרת קוד משפת C לקובץ הרצה EXE

באותו אופן, גם הקובץ הבינארי של `my_funcs` מכיל את פקודות האסמבלי, אך עם כתובות לא מפוענחות בזיכרון:

seg000:000000DF	8B 55 08	mov	edx, [ebp+8]
seg000:000000E2	8B 45 0C	mov	eax, [ebp+0Ch]
seg000:000000E5	01 C2	add	edx, eax
seg000:000000E7	A1 00 00 00 00	mov	eax, ds:dword_0
seg000:000000EC	01 D0	add	eax, edx

שימו לב לכתובות בזיכרון של הקוד, הכתובות היחסיות לתחילת הסגמנט נמצאות בין xDF0 לבין xEC0. למרות שבמקרה אין חפיפה עם הכתובות של הקובץ השני, יכל בהחלט להיות מצב שבו שני הקבצים מקבלים אזורי כתובות חופפים עבור הקוד שלהם. אי אפשר לטעון את הקבצים הבינאריים לזיכרון כפי שהם.

תהליך המרת קוד משפת C לקובץ הרצה EXE

תהליך ה-Linking

מי שתפקידו לספק את הכתובות החסרות הוא ה-Linker. תהליך ה-Linking מאחד את כל אזורי הקוד לאזור אחד ואת כל אזורי הנתונים לאזור אחד. כעת, כאשר ברור איזו פקודה מגיעה אחרי איזו פקודה ומהם כל המשתנים שצריך להקצות להם זיכרון, אפשר לחלק כתובות וירטואליות לכל פונקציה ומשתנה.

תהליך המרת קוד משפת C לקובץ הרצה EXE

.text:00401410 55	push	ebp
.text:00401411 89 E5	mov	ebp, esp
.text:00401413 83 E4 F0	and	esp, 0FFFFFFF0h
.text:00401416 83 EC 20	sub	esp, 20h
.text:00401419 E8 62 05 00 00	call	__main
.text:0040141E 8B 15 04 40 40 00	mov	edx, _b_2064
.text:00401424 A1 08 40 40 00	mov	eax, _a_2063
.text:00401429 89 54 24 04	mov	[esp+4], edx
.text:0040142D 89 04 24	mov	[esp], eax
.text:00401430 E8 1F 00 00 00	call	_my_add
.text:00401435 89 44 24 1C	mov	[esp+1Ch], eax
.text:00401439 8B 44 24 1C	mov	eax, [esp+1Ch]
.text:0040143D 89 44 24 04	mov	[esp+4], eax
.text:00401441 C7 04 24 44 50 40+	mov	dword ptr [esp], offset aD ; "%d"
.text:00401448 E8 73 26 00 00	call	_printf
.text:0040144D B8 00 00 00 00	mov	eax, 0
.text:00401452 C9	leave	
.text:00401453 C3	retn	
.text:00401453	__main	endp
.text:00401453		
.text:00401454		

תהליך המרת קוד משפת C לקובץ הרצה EXE

```
.text:00401454      ; ===== S U B R O U T I N E =====
.text:00401454
.text:00401454      ; Attributes: bp-based frame
.text:00401454
.text:00401454      public _my_add
.text:00401454      _my_add      proc near      ; CODE XREF: _main+20↑p
.text:00401454
.text:00401454      arg_0        = dword ptr 8
.text:00401454      arg_4        = dword ptr 0Ch
.text:00401454
.text:00401454 55                push     ebp
.text:00401455 89 E5            mov      ebp, esp
.text:00401457 8B 55 08         mov      edx, [ebp+arg_0]
.text:0040145A 8B 45 0C         mov      eax, [ebp+arg_4]
.text:0040145D 01 C2           add      edx, eax
.text:0040145F A1 0C 40 40 00   mov      eax, _one_1554
.text:00401464 01 D0           add      eax, edx
.text:00401466 5D              pop      ebp
.text:00401467 C3              retn
.text:00401467      _my_add      endp
```

תהליך המרת קוד משפת C לקובץ הרצה EXE

נדון בשלושת החלקים שקבע ה-Linker: הכתובות של הפקודות, הכתובות של המשתנים, הכתובות של הפונקציות.

כתובות הפקודות

```
.text:0040141E 8B 15 04 40 40 00      mov     edx, _b_2064  
.text:00401424 A1 08 40 40 00      mov     eax, _a_2063
```

הפקודה `mov edx, _b_2064`

נמצאת בכתובת 0x0040141E. זהו אזור זיכרון שמוגדר בתור text, כלומר קוד. יש לו הרשאות מתאימות (קריאה והרצה). הפקודה הבאה נמצאת בכתובת 0x00401424, כלומר 6 בתים לאחר תחילת הפקודה הקודמת. אם תיזכרו בתרגיל הקטן שביצענו כאשר למדנו על מבנה כתובות וירטואליות, בו פירקנו את הכתובת 0x00401000 למרכיביה השונים, תשימו מיד לב לכך שה-Linker קבע לפקודות כתובות וירטואליות.

תהליך המרת קוד משפת C לקובץ הרצה EXE

אנחנו עדיין מסתכלים בקוד כפי שהוא שמור בקובץ על הדיסק הקשיח. יש אפשרות שלאחר שהקוד ייטען ל-RAM יוקצו לו כתובות וירטואליות אחרות. כלומר יכול להיות שהפקודה הראשונה תיטען לכתובת 0x0050141E. אך דבר אחד יישאר קבוע- ההפרש של הבתים בין הפקודות. תהליך הטעינה לוקח את כל הפקודות וקובע להן כתובות וירטואליות חדשות או מותיר אותן במקום שבו ה-Linker המליץ לשים אותן, אך בכל מקרה כל הפקודות זזות יחד כקבוצה.

תהליך המרת קוד משפת C לקובץ הרצה EXE

כתובות המשתנים

נסקור את הכתובות הוירטואליות שקבע ה-Linker למשתנים שלנו

.text:0040141E 8B 15 04 40 40 00	mov	edx, _b_2064
.text:00401424 A1 08 40 40 00	mov	eax, _a_2063
.text:0040145F A1 0C 40 40 00	mov	eax, _one_1554

Little Endian

המשתנה	כתובת וירטואלית
b	0x00404004
a	0x00404008
one	0x0040400C

תהליך המרת קוד משפת C לקובץ הרצה EXE

המשתנה	כתובת וירטואלית
b	0x00404004
a	0x00404008
one	0x0040400C

חשוב להבין שלמרות שה-Linker העניק כתובות לכל המשתנים, אנחנו עדיין לא יודעים מה הכתובות שייעשה בהן שימוש כאשר הקובץ ייטען ל-RAM. אפשר להתייחס אל הכתובות שה-Linker קבע בתור "המלצות". מה שכן יישמר הוא היחסיות ביניהן. כלומר אם ה-Linker קבע שמשתנה a נמצא בכתובת 0x00404008 ואילו המשתנה b נמצא בכתובת 0x00404004, אז הפער ביניהם בגודל 4 בתים יישמר גם אם הן יוזזו ממקומם בהמשך.

תהליך המרת קוד משפת C לקובץ הרצה EXE

כתובות הפונקציות

```
.text:00401430 E8 1F 00 00 00      call     _my_add
.text:00401448 E8 73 26 00 00      call     _printf
```

הכתובות שהוענקו לפונקציות הן כתובות יחסיות לפקודה שמבצעת להן call, כלומר הן בעצם מבטאות כמה בתים צריך לקפוץ כדי להגיע לפונקציה:

כתובת יחסית	הפונקציה
0x0000001F	my_add
0x00002673	printf

תהליך המרת קוד משפת C לקובץ הרצה EXE

```
.text:00401430 E8 1F 00 00 00 call _my_add
```

לדוגמה, הקריאה לפונקציה `my_add` מתבצעת בפקודה שנמצאת בכתובת `0x00401430` וגודלה 5 בתים. כלומר כשהפקודה הזו מורצת הרגיסטר EIP כבר מצביע על הכתובת של הפקודה הבאה אותה צריך להריץ, `0x00401435`. הקריאה לפונקציה אומרת "הזז את EIP כ-`0x1F` בתים מהמקום בו הוא נמצא כרגע". ההזזה הזו, אם תבצעו את החישוב, מעלה את ערכו של EIP להיות `0x00401454`. זו בדיוק הכתובת הוירטואלית בה נמצאת הפונקציה `my_add`.

```
.text:00401454 public _my_add
```

תהליך המרת קוד משפת C לקובץ הרצה EXE

הבנו את התהליך שעבר הקוד שכתבנו עד להפיכתו לקובץ exe, אולם יש קטע קוד אחד שלא אנחנו כתבנו ולמרות זאת מופיע בקוד שלנו- הפונקציה printf.

.text:00401448 E8 73 26 00 00 call _printf

הפונקציה printf אמורה להיות כ-0x2673 בתים מהמקום בו היא נקראת, כלומר בכתובת 0x00403AC0. נבדוק מה יש בכתובת זו:

```
.text:00403AC0          ; ===== S U B R O U T I N E =====
.text:00403AC0
.text:00403AC0          ; Attributes: thunk
.text:00403AC0
.text:00403AC0          ; int printf(const char *, ...)
.text:00403AC0          public _printf
.text:00403AC0          _printf      proc near          ; CODE XREF: _main+38↑p
.text:00403AC0 FF 25 D8 81 40 00      jmp     ds:__imp__printf
.text:00403AC0          _printf      endp
.text:00403AC0
.text:00403AC0          ; -----
```

תהליך המרת קוד משפת C לקובץ הרצה EXE

```
.text:00403AC0          ; ===== S U B R O U T I N E =====
.text:00403AC0
.text:00403AC0          ; Attributes: thunk
.text:00403AC0
.text:00403AC0          ; int printf(const char *, ...)
.text:00403AC0          public _printf
.text:00403AC0          _printf      proc near                ; CODE XREF: _main+38↑p
.text:00403AC0 FF 25 D8 81 40 00      jmp     ds:__imp__printf
.text:00403AC0          _printf      endp
.text:00403AC0
.text:00403AC0          ; -----
```

אנחנו רואים קפיצה נוספת. הפעם זו אינה קפיצה יחסית אלא קפיצה ממש, לכתובת 0x004081D8

ניגש לכתובת הזו

תהליך המרת קוד משפת C לקובץ הרצה EXE

שוב לא נמצא את הקוד שלה. במקום זאת, נגיע אל אזור זיכרון בשם idata ונמצא את המידע הבא:

```
.idata:00408180 ;  
.idata:00408180 ; Imports from msvcrt.dll  
.idata:00408180 ;  
  
.idata:004081D8 ; int _printf(const char *, ...)  
.idata:004081D8          extrn __imp__printf:dword
```

כלומר הפונקציה printf מיובאת מתוך קובץ DLL בשם msvcrt.dll. הקובץ הנ"ל הוא קוד מקומפל של מיקרוסופט, שבתוכו יש פונקציות שימושיות כגון הדפסה, הקצאה ושחרור של זיכרון, טיפול במחרוזות. אפשר למצוא בתוך IDA את כל הפונקציות שייבאנו מתוך msvcrt.dll. בהמשך הפרק נעסוק בקבצי DLL, נלמד ליצור אותם ונראה מה קורה לקבצי DLL בזמן תהליך ה-RAM Loading. בשלב זה מה שאנחנו מבינים, הוא שה-Linker לא יודע מה המיקום בזיכרון של הפונקציה printf, אלא הוא סומך על מישהו אחר שישלים את הכתובת הזו. את הפרטים המלאי נבין בפרק הבא, כאשר נלמד על תהליך ה-RAM Loading ועל ה-Import Address Table.

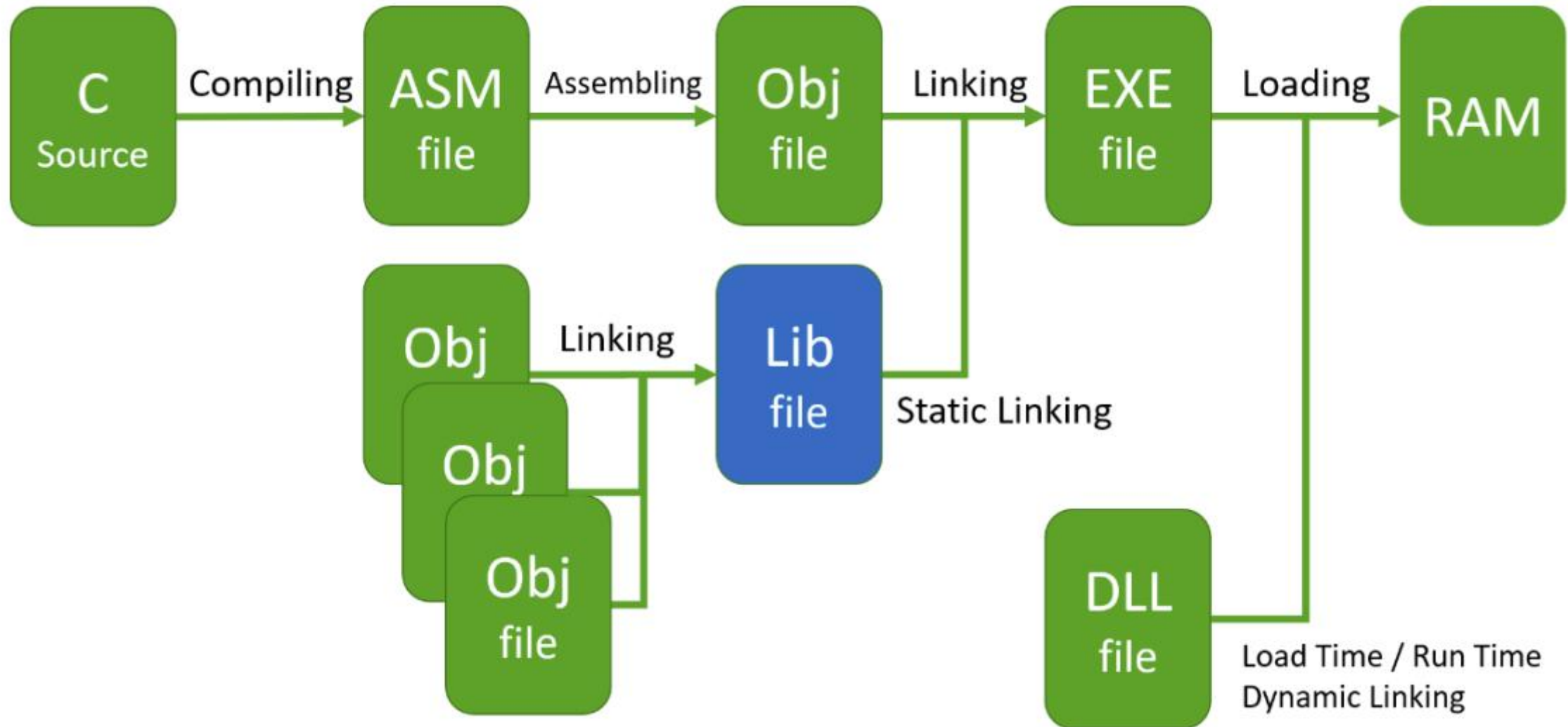
יצירת LIBRARY וביצוע STATIC LINKING

יצירת LIBRARY וביצוע STATIC LINKING

לפני הכל, חשוב להדגיש שהמונחים והשרטוטים מכאן והלאה נכונים ל-Windows. במערכות הפעלה אחרות יש הבדלים, לדוגמה ב-Linux אין קבצי lib אלא קבצים בסיומת a ובמקום קבצי DLL יש קבצים בסיומת so. אך באופן כללי העקרונות דומים ומי ששולט בתהליכים שעובר קוד ב-Windows יוכל לגשר על ההבדלים ולהבין גם מערכות הפעלה שונות.

ב-Windows, קבצי lib (קיצור של Library) מציינים קוד מקומפל, שעדיין לא עבר את השלב האחרון הנדרש להפיכתו לקוד הרצה exe. בדומה לקובץ obj שסקרנו בסעיף הקודם, הקוד של ה-Library הוא בשפת מכונה, אך המעבד לא יכול להריץ אותו כיוון שלא בוצע לו Linking: הכתובות שרשומות בקוד (פונקציות, משתנים) אינן נכונות. אם יש ברשותנו Library כלשהי, אנחנו יכולים להפוך אותה לקוד הרצה באמצעות פעולת Linking עם קוד אחר שיש לנו. כך יהפוך ה-Library לחלק מקוד התוכנית שלנו, ממש כאילו כתבנו אותו בעצמנו.

יצירת LIBRARY וביצוע STATIC LINKING



יצירת LIBRARY וביצוע STATIC LINKING

אם כך, מה ההבדל בין Library לקובץ obj? ה-Library הוא תוצר של אחד או יותר קבצי obj שבוצע להם Linking זה עם זה. היתרון של קובץ lib יחיד על פני מספר קבצי obj הוא בכך שישנה טבלה אחת של סימבולים לכל הקובץ. כאשר ה-Linker מבצע את השלב האחרון של המרה לקובץ exe הוא אינו צריך לחפש כל סימבול שאינו מוכר לו בכל קבצי ה-obj האפשריים, אלא רק בטבלה אחת.

בעבר, טרם פיתוח ה-DLLים, הצורך ב-Library נבע מכך שיש קטעי קוד שהם שימושיים למספר רב של תוכנות. לדוגמה, קוד שיועד לקלוט היכן המשתמש הקליק עם העכבר על המסך, יהיה שימושי לכל תוכנה שיש לה חלון משתמש. לכן כל מי שמתכנת תוכנה שיש לה חלון משתמש ירצה לשלב את הקוד הזה בתוכנה שלו. אפשרות אחת היא לקחת את הקוד שמטפל בעכבר ולקמפל אותו יחד עם קוד התוכנה. כך ניצור קובץ הרצה exe שיש בו כבר את כל הפונקציות שמטפלות בעכבר.

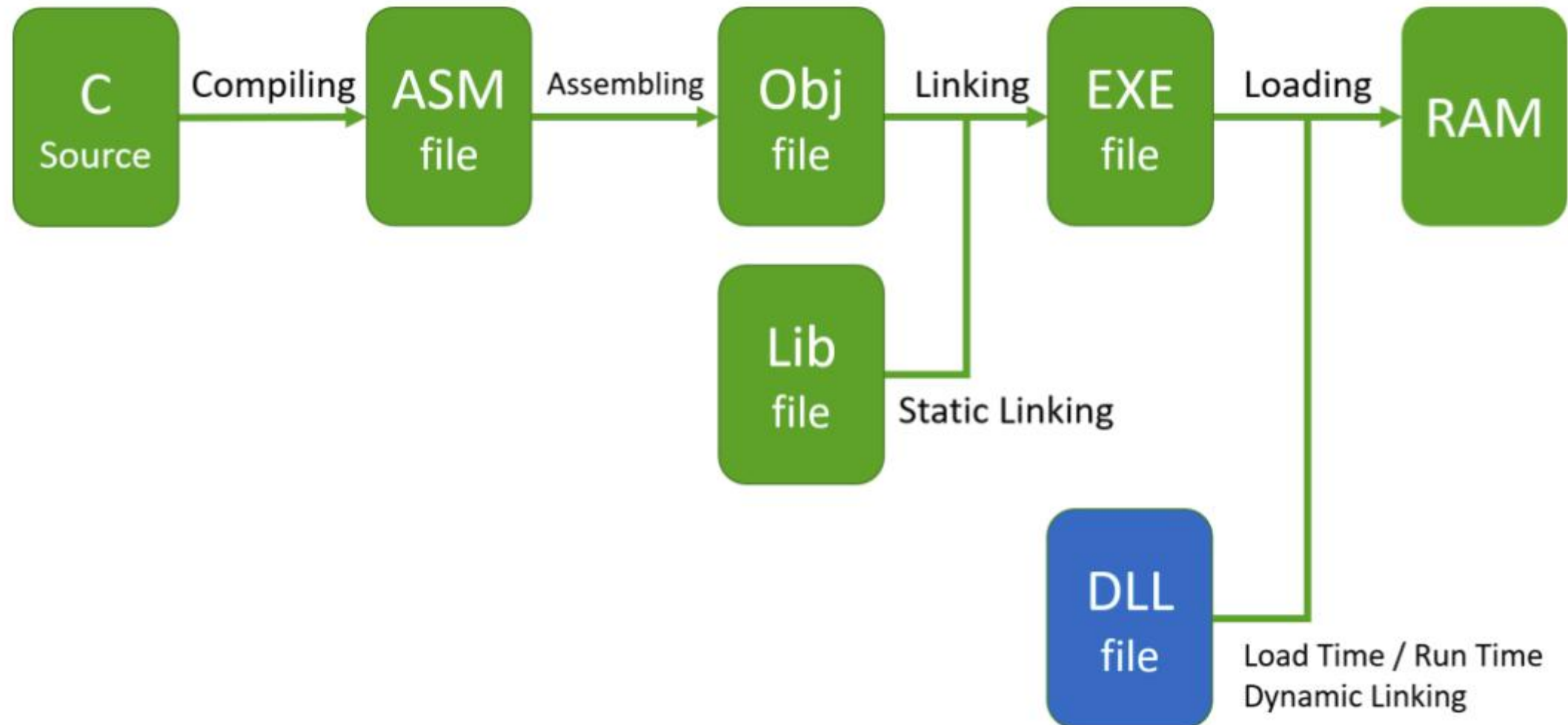
יצירת LIBRARY וביצוע STATIC LINKING

אולם בגישה זו יש בעיה: אנחנו צריכים את קוד המקור, ואילו חברות תוכנה אינן מספקות את קוד המקור שלהן, בדרך כלל, מסיבות של סודיות מסחרית. לכן נדרשת שיטה שתאפשר לקמפל קוד שלנו יחד עם קוד של מישהו אחר – מבלי להפר או לפגוע באותה סודיות.

על הבעיה הזו ענה שימוש בקבצי lib. היכנסו אל Visual Studio ופיתחו פרוייקט כלשהו. שנו את הגדרות הפרוייקט כך שהתוצר שלו יהיה קובץ lib. כאשר תבצעו build, תיצרו קובץ lib. את קובץ ה-lib שיצרתם אתם יכולים למסור לכל מי שתמצו שיעשה בו שימוש, והוא יבצע Linking אל הקוד שלו. צורה זו של Linking נקראת **Static Linking**. היתרון שלה הוא שהוא מאפשר לנו לחלוק חבילות קוד, בלי לחלק את קוד המקור שעמלנו רבות על פיתוחו. בעבר, זו היתה הצורה היחידה של שיתוף קוד.

קובץ DLL

בחלק זה נענה על השאלה מהו DLL ומדוע יש בו צורך, ונסביר מה ההבדל בין Load Time Dynamic Linking ו-Run Time Dynamic Linking.



קובץ DLL

השם DLL הוא ראשי תיבות של Dynamic Link Library. הפורמט של קבצי DLL הומצא על ידי מיקרוסופט, אם כי הרעיון הכללי נמצא גם במערכות הפעלה אחרות. ב-Linux יש קבצים שמשמשים לתכלית דומה, אך הם נקראים קבצי SO, קיצור של Shared Object. הצורך ב-Dynamic Linking נובע מכך שישנן בעיות ש-Static Linking אינו פותר.

הבעיה הראשונה ב-Static Linking היא קושי בתחזוקה של קוד. נניח שחברת התוכנה שכותבת קוד שמטפל בעכבר שיפרה את הקוד. כעת כל מי שרוצה ליהנות מהשיפור צריך לבצע Linking מחדש של התוכנה שלו עם ה-library המשודרגת ולהפיץ לכל המשתמשים שלו את העדכון. תהליך מסובך ולא יעיל. בנוסף, ביצוע Linking לספריות קוד מגדיל את גודלו של קובץ ההרצה, ה-exe, ועלול לבזבז משאבים. לדוגמה מקום בדיסק הקשיח או כמה טוב היה אם היה מספיק רק לעדכן קובץ, שכולל את הקוד המשודרג של העכבר, וכל התוכנות במחשב שמשתמשות בקובץ הזה יקבלו מיידית את היכולות המשודרגות.

קובץ DLL

הבעיה השניה היא בזבוז של זיכרון. אם עשר תוכנות נמצאות בזיכרון שלנו, וכולן עושות שימוש בעכבר, אז הפונקציות של העכבר משוכפלות עשר פעמים בזיכרון. מדוע שהקוד שמטפל בעכבר לא ייטען פעם אחת לזיכרון, וכל מי שזקוק לו יקרא אותו משם?

זהו היתרון של DLL. זהו קוד, שמכיל פונקציות שונות. הפונקציות הללו נטענות לזיכרון פעם אחת, וכל התוכנות שרצות במקביל משתמשות באותו עותק שנמצא בזיכרון. לכן, מספיק שיש לנו עותק אחד של ה-DLL על המחשב, שנטען לזיכרון פעם אחת. אם יש לנו עדכונים לתוכנה, נפיץ DLL חדש, כל מי שמשתמש בתוכנה שלנו יחליף את ה-DLL הישן, וכל התוכנות שעושות בו שימוש יקבלו מיד את היכולות החדשות.

הדוגמה שניתנה לפני כן, של קוד שמפעיל את העכבר, לא היתה מנותקת ממה שמתרחש באמת. פונקציות שמנהלות את העכבר, כמו פונקציות נוספות שעובדות מול חלונות פתוחים, נמצאות בקובץ user32.dll.

קובץ DLL

קיימות שתי דרכים לטעון DLL לתוך תוכנה - Load Time Dynamic Linking ו- Run Time Dynamic Linking.

ב- **Load Time Dynamic Linking** ה-DLL המבוקש נטען עם טעינת התוכנה. למה הכוונה? בניגוד ל-Static Linking, הקוד של ה-exe שנוצר אינו כולל את קוד ה-Library. קוד ה-Library נשאר בקובץ נפרד, מסוג DLL. יכול להיות exe שמשתמש ב-DLL שגודלו 1MB, ועדיין גודלו של ה-exe יהיה מספר KB בודדים.

איך מתבצע ה-Linking בין ה-exe וה-DLL, שהינם שני קבצים נפרדים? לכל קובץ הרצה exe יש חלק, שבו כתובים כל ה-DLLים שצריך לטעון לזיכרון לפני שהמעבד מתחיל להריץ את התוכנה עצמה. לכן השיטה בה הטעינה של ה-DLLים שרשומים בקובץ ה-exe מתרחשת בזמן טעינת ה-exe, נקראת Load Time Dynamic Linking. בזמן הטעינה, ה-DLL ממופה לתוך הזיכרון הוירטואלי של ה-Process שנוצר, כך שניתן יהיה לקרוא לפונקציות שלו. אם יש כבר עותק של ה-DLL המבוקש ב-RAM לא יהיה צורך לטעון אותו שוב, אלא רק לגרום להעביר ל-Process את הכתובות של ה-DLL ושל הפונקציות הנדרשות. בכל מקרה ה-DLL ייטען אל ה-RAM רק פעם אחת, וכל מי שמשתמש בו יעשה שימוש בעותק היחיד הטעון ל-RAM.

קובץ DLL

קיימות שתי דרכים לטעון DLL לתוך תוכנה- Load Time Dynamic Linking ו- Run Time Dynamic Linking.

ב- **Run Time Dynamic Linking** הרעיון מאד דומה, אך בהבדל אחד מרכזי: אמרנו שקובץ ה-exe כולל את רשימת ה-DLLים שיש למפות לזיכרון בזמן הטעינה. כצפוי, DLL שאינו נמצא ברשימה זו לא ייטען. אבל אפשר שבזמן הריצה ייקראו פונקציות שיטענו בעצמן את ה-DLL הנדרש. השיטה שבה ה-DLL נטען בזמן ריצה נקראת Run Time Dynamic Linking.

בסעיפים הבאים נלמד כיצד יוצרים DLL וכיצד טוענים אותו, בשתי השיטות.

קובץ DLL

יצירת DLL

נפתח פרוייקט חדש, בדוגמה בחרנו לקרוא לו mydll.

בתור התחלה, נוסיף לפרוייקט שלנו קובץ h חדש (בתוך Solution Explorer קליק ימני על Header Files ואז Add, בחרו New Item ולאחר מכן Header File). נקרא לקובץ mydll.h. נכתוב לתוכו את הקוד הבא:

```
#pragma once
```

```
#ifdef DLL_EXPORT
```

```
#define DECLDIR __declspec(dllexport)
```

```
#else
```

```
#define DECLDIR __declspec(dllimport)
```

```
#endif
```

```
extern "C"
```

```
{
```

```
    DECLDIR void Share();
```

```
    void Keep();
```

```
}
```

כהעשרה, מומלץ לקרוא על ההנחיות לקומפילר (pragma once, ifdef, extern "C") – לא בחומר הקורס

קובץ DLL

```
DECLDIR void Share();  
void Keep();
```

נתמקד בשתי הפונקציות שמוגדרות - Share ו-Keep.

לפני הכל, חשוב להבין שכאשר אנחנו מפיצים DLL הכולל פונקציות שונות, ישנן פונקציות שנרצה שלא ייעשה בהן שימוש על ידי מי שאנחנו מוסרים לו את ה-DLL. אם הקוד שלנו כולל פונקציות בעלות יכולות מיוחדות, שאנחנו רוצים שיישארו רק ברשותנו, נצטרך דרך לסמן אילו פונקציות פתוחות לשימוש של כולם ואילו לא.

לפונקציה Share קודמת ההגדרה DECLDIR, שהפרוש שלה משתנה אם מוגדר DLL_EXPORT או לא. בהנחה שכן, אז הקידומת תפורש כ- `__declspec(dllexport)` כלומר הפונקציה תיוצא על ידי ה-DLL. המשמעות של זה היא שה-DLL יחשוף את הפונקציה הזו לשימוש של כל מי שמבצע linking ל-DLL.

לעומתה, הפונקציה Keep היא בשום מקרה לא מיוצאת, ולכן הקוד היחיד שיכול להשתמש בה הוא קוד של ה-DLL עצמו.

קובץ DLL

לאחר שהגדרנו את קובץ ה-header, נעבור לקובץ הראשי- mydll.cpp.

```
#include "pch.h"  
#define DLL_EXPORT  
#include "mydll.h"
```

בתחילת הקובץ ניתנות מספר הנחיות לקומפיילר:

1. Include לקובץ pch.h-

(משמש ל-Include לקבצים שאינם משתנים וכך חוסך זמן קומפילציה)

2. הגדרת DLL_EXPORT, שהיא משמעותית לטובת ההגדרות בקובץ mydll.h ולכן הכרחי שתופיע לפניו. שימו לב לא לשים את ההגדרה לפני ה-Include לקובץ pch.h, איננו יודעים באיזה מקום עלול להיות תנאי שקשור להנחיה זו!

3. Include לקובץ mydll.h

```
extern "C"
```

```
{
```

```
DECLDIR void Share()
```

```
{
```

```
    printf("I am an exported function, can be called outside  
           the DLL\n");
```

```
}
```

```
void Keep()
```

```
{
```

```
    printf("I am not exported, can be called only within the  
           DLL\n");
```

```
}
```

```
}
```

לאחר מכן מוגדרות הפונקציות ו-Keep

לאחר מכן מגיע החלק המעניין - dllmain

```
BOOL APIENTRY DllMain(HANDLE hModule,  
module
```

```
        DWORD ul_reason_for_call,  
        LPVOID lpReserved)    // Reserved
```

```
{
```

```
    switch (ul_reason_for_call)
```

```
    {
```

```
    case DLL_PROCESS_ATTACH:
```

```
        // A process is loading the DLL.
```

```
        Share();
```

```
        Keep();
```

```
        break;
```

```
case DLL_THREAD_ATTACH:  
    // A process is creating a new thread.  
    break;
```

```
case DLL_THREAD_DETACH:  
    // A thread exits normally.  
    break;
```

```
case DLL_PROCESS_DETACH:  
    // A process unloads the DLL.  
    break;
```

```
}
```

```
return TRUE;
```

```
}
```

קובץ DLL

לאחר מכן מגיע החלק המעניין- `dllmain`. זוהי נקודת הכניסה לתוך ה-DLL שלנו. אם יש קוד שאנחנו רוצים שירץ אוטומטית, זה המקום להכניס אותו. חשוב להבין: קוד שנמצא ב-`dllmain` ירוץ גם בלי שהתוכנה הראשית, שטוענת את ה-DLL, תקרא לו. מערכת ההפעלה קוראת ל-`dllmain` והיא עושה זאת במספר מצבים:

```
switch (ul_reason_for_call)
```

1. Process טען את ה-DLL שלנו `case DLL_PROCESS_ATTACH:`

2. ה-Process שטען את ה-DLL שלנו יוצר Thread חדש `case DLL_THREAD_ATTACH:`

3. Thread שה-DLL שלנו היה טעון בו, מסיים ריצה `case DLL_THREAD_DETACH:`

4. Process שטען את ה-DLL שלנו מבצע Unload, הסרת ה-DLL שלנו

```
case DLL_PROCESS_DETACH:
```

קובץ DLL

לאחר מכן מגיע החלק המעניין- `dllmain`. זוהי נקודת הכניסה לתוך ה-DLL שלנו. אם יש קוד שאנחנו רוצים שירוץ אוטומטית, זה המקום להכניס אותו. חשוב להבין: קוד שנמצא ב-`dllmain` ירוץ גם בלי שהתוכנה הראשית, שטוענת את ה-DLL, תקרא לו. מערכת ההפעלה קוראת ל-`dllmain` והיא עושה זאת במספר מצבים:

כל אחד מהמקרים האלה מעביר את השליטה ל-`dllmain`. ההגיון בכך, הוא שאם תוכנה מסויימת טוענת את ה-DLL שלנו, ייתכן ונרצה להריץ פעולות מקדימות כלשהן, שיאפשרו לפונקציות שלנו לרוץ היטב. לדוגמה,

בדיקה מהי החומרה שאנחנו רצים עליה, הקצאת זיכרון וכו'. גם בסיום ריצה או הסרת ה-DLL שלנו ייתכן שיש פעולות שאנחנו צריכים לבצע, לדוגמה שחרור זיכרון, שמירת מידע לקובץ ועוד. חשוב מאד לשים לב שמיקרוסופט מגבילים מאד את הקוד שניתן להריץ אוטומטית מ-`dllmain`, דברים מסויימים עלולים לגרום לקריסת ה-Process שטען את ה-DLL.

קובץ DLL

בקוד הדוגמה שלנו, אנחנו משתמשים רק באפשרות להריץ קוד עם טעינת ה-DLL. מתבצעת קריאה לפונקציה Share ולאחר מכן קריאה לפונקציה Keep, שנועדה להמחיש שהקוד של ה-DLL יכול לקרוא ל-Keep ללא קושי. הפונקציה Keep חסומה רק לקריאות מקוד חיצוני.

נותר לנו רק לייצר את ה-DLL. כדי לעשות זאת, נשנה את הגדרות הקונפיגורציה של הפרוייקט שלנו, כך שבמקום EXE יוצר DLL:

mydll Property Pages

Configuration: Active(Release)

Platform: Active(Win32)

Configuration Properties

General

Debugging

VC++ Directories

▷ C/C++

▷ Linker

▷ Manifest Tool

▷ XML Document Generator

▷ Browse Information

▷ Build Events

▷ Custom Build Step

▷ Code Analysis

General

Target Platform	Windows 10
Windows SDK Version	10.0.17134.0
Output Directory	\$(SolutionDir)\$(Configuration)\
Intermediate Directory	\$(Configuration)\
Target Name	\$(ProjectName)
Target Extension	.dll
Extensions to Delete on Clean	*.cdf;*.*cache;*.*obj;*.*obj.enc;*.*ilk;*.*ipdb;*.*iob*
Build Log File	\$(IntDir)\$(MSBuildProjectName).log
Platform Toolset	Visual Studio 2017 (v141)
Enable Managed Incremental Build	No

Project Defaults

Configuration Type	Dynamic Library (.dll)
Use of MFC	Use Standard Windows Libraries
Character Set	Use Unicode Character Set
Common Language Runtime Support	No Common Language Runtime Support
.NET Target Framework Version	
Whole Program Optimization	Use Link Time Code Generation
Windows Store App Support	No

שינוי הגדרות הפרוייקט כך שיווצר DLL

קובץ DLL

הידד! יש לנו DLL. אם נפתח את תיקיית ה-Release של הפרוייקט mydll, נוכל למצוא שם את הקובץ mydll.dll וכן את הקובץ mydll.lib. מדוע נוצר לנו קובץ lib? הרי אנחנו מבצעים Dynamic Linking, ואמרנו שקבצי lib משמשים עבור Static Linking? התשובה היא, שהסיומת היא אותה סיומת אך השימוש שונה. בניגוד ל-Static Linking, קובץ ה-lib שנוצר לנו אינו כולל קוד, אלא רק אזכור של הפונקציות שה-DLL מייצא. כאשר אנחנו יוצרים קובץ EXE שקורא לפונקציות מתוך DLL, ה-linker משתמש בקובץ ה-lib כדי לשתול בקובץ ההרצה הפניות מתאימות אל ה-DLL, כך שכל פעם שנקרא לפונקציה מתוך ה-DLL תבוצע קפיצה אל הקוד שב-DLL. תהליך זה מאפשר לנו לקרוא לפונקציות שב-DLL ממש כאילו הן פונקציות של הקוד שלנו. מיד נראה כיצד משתמשים בקובץ ה-lib.

קובץ DLL

יבוא DLL ב-Load Time

ניצור פרוייקט חדש, בדוגמה שלנו usemydll.

נערוך את הקובץ usemydll.cpp באופן הבא:

```
#include "pch.h"

extern "C" __declspec(dllimport) void Share();
// Other option would be-
// #include "mydll.h"

int main()
{
    Share();
    return 0;
}
```

קובץ DLL

מבין שתי הפונקציות המוגדרות ב-DLL, אפשר לקרוא מהקוד שלנו רק לפונקציה `Share`. כזכור, הפונקציה `Keep` אינה מיוצאת. כדי לשלב את הפונקציה `Share` אנחנו צריכים למסור לקומפיילר מידע על המבנה שלה. את המידע הזה אפשר למסור בשתי דרכים. דרך אחת היא לבצע `include` לקובץ `mydll.h`. זוהי הדרך הקלה, אך היא דורשת שהקובץ `mydll.h` יהיה ברשותנו. לא בטוח שזה יהיה המצב. הדרך השנייה היא, כפי שנכתב בקוד שלנו, לתת לקומפיילר את המבנה של `Share` ולומר לו שזוהי פונקציה מיובאת.

נותר לנו רק לשנות את הגדרות הפרוייקט.

כדי להודיע ללינקר היכן הוא יכול למצוא את mydll.dll, נכניס את התיקיה של mydll.dll לתוך משתנה הסביבה PATH. בתוך Environment נכתוב:

`PATH=mydll_path;%PATH%`

כאשר mydll_path הוא הנתיב המלא אל ה-DLL שלנו. לדוגמה

`PATH=c:\software\dlls;%PATH%`

usemydll Property Pages

Configuration: Active(Debug) Platform: Active(Win32) Con

- Configuration Properties
 - General
 - Debugging**
 - VC++ Directories
- C/C++
- Linker
 - General
 - Input
 - Manifest File
 - Debugging
 - System

Debugger to launch:

Local Windows Debugger

Command	\$(TargetPath)
Command Arguments	
Working Directory	\$(ProjectDir)
Attach	No
Debugger Type	Auto
Environment	PATH=C:\Users\barak\source\repos\mydll\Release;%PATH%
Merge Environment	Yes
SOL Debugging	No

קובץ DLL

בשלב האחרון אנחנו צריכים לסייע ללינקר למצוא את קובץ ה-lib mydll.lib. נערוך את השדה Additional Dependencies כך שיכלול את שם ה-lib ואת הנתיב המלא אליו:

Configuration: Active(Debug) Platform: Active(Win32)

Configuration Properties	Additional Dependencies	C:\Users\barak\source\repos\mydll\Release\mydll.lib;%
General	Ignore All Default Libraries	
Debugging	Ignore Specific Default Libraries	
VC++ Directories	Module Definition File	
▸ C/C++	Add Module to Assembly	
▴ Linker	Embed Managed Resource File	
General	Force Symbol References	
Input	Delay Loaded DLLs	
Manifest File	Assembly Link Resource	

קובץ DLL

זהו!

נעת נריץ את התוכנית usemydll.

בעקבות הרצת usemydll תופיע ההדפסה הבאה:

Microsoft Visual Studio Debug Console

```
I am an exported function, can be called outside the DLL  
I am not exported, can be called only within the DLL  
I am an exported function, can be called outside the DLL
```

בעקבות הרצת usemydll תופיע ההדפסה הבאה:

```
Microsoft Visual Studio Debug Console

I am an exported function, can be called outside the DLL
I am not exported, can be called only within the DLL
I am an exported function, can be called outside the DLL
```

מה גרם להדפסת הפלט הזה?

שתי השורות הראשונות הן תוצאה של טעינת ה-DLL. כזכור, בתוך `dllmain` יש קריאה ל-`Share` ול-`Keep`. השורה השלישית היא תוצאה של הקריאה ל-`Share` מתוך הקוד של `usemydll`.

```
case DLL_PROCESS_ATTACH:
    // A process is loading the DLL.
    Share();
    Keep();
    break;
```

```
int main()
{
    Share();
    return 0;
}
```

קובץ DLL

יבוא DLL ב- Run Time

בניגוד ל-Load Time Dynamic Linking, כעת אנחנו לא מוסרים ללינקר שלנו פרטים על ה-DLL. אילו פרטים העברנו ללינקר ב-Load Time וכעת חסרים? העברנו לו את:

1. מיקום ה-DLL שאנחנו רוצים לטעון
2. קובץ ה-lib שכולל את שמות הפונקציות

קובץ DLL

כתחליף ל-א', נשתמש בפונקציה LoadLibrary. הפונקציה מקבלת נתיב מלא הכולל שם של DLL, טוענת את ה-DLL לזיכרון ומחזירה מצביע על ה-DLL המבוקש (אם לא אירעה שגיאה).

```
HMODULE hModule = LoadLibraryA(LIBRARY);  
if (NULL == hModule) {  
    printf("Failed to load DLL\n");  
    return 0;  
}
```

כתחליף ל-ב', נשתמש בפונקציה GetProcAddress. כפי שאפשר להבין משמה, היא מחזירה את הכתובת בזיכרון בה נמצאת הפרוצדורה שביקשנו (בשפת אסמבלי, פרוצדורה הינה פונקציה).

```
PFUNC pFunc = (PFUNC)GetProcAddress(hModule, "Share");
```

קובץ DLL

```
typedef void(*PFUNC)(void);
```

שימו לב לטכניקה שבה אנחנו משתמשים בכתובת החזרה. אנחנו מודיעים לקומפיילר, באמצעות typedef, ש-PFUNC הוא מצביע לפונקציה, שמקבלת void ומחזירה void. זו צורת הכתיבה המקובלת ב-C.

```
PFUNC pFunc = (PFUNC)GetProcAddress(hModule, "Share");
```

כאשר אנחנו קוראים ל-GetProcAddress, אנחנו מבצעים Casting של ערך החזרה: ערך החזרה הוא כאמור כתובת בזיכרון. ה-Casting מסביר לקומפיילר, שהכתובת הזו היא מצביע לפונקציה, ושהפונקציה הזו מקבלת void ומחזירה void.